
MULTI-LAYER SECURITY TESTING DASHBOARD
MULTI-LAYER SECURITY TESTING DASHBOARD
MULTI-LAYER SECURITY TESTING DASHBOARD

ABSTRACT

The Multi-Layer Security Testing Dashboard is a web-based platform designed to detect major web application vulnerabilities. Built using Flask, HTML/CSS/JS, and Supabase PostgreSQL, it tests five key OWASP vulnerabilities: SQL Injection, XSS, Broken Access Control, AI Prompt Injection, and Cryptographic Failures.

It includes JWT-based authentication, real-time detection, automated testing, and a centralized dashboard with logs stored in the cloud. This system serves as both an educational tool and a practical testing platform for developers to analyze security weaknesses over time.

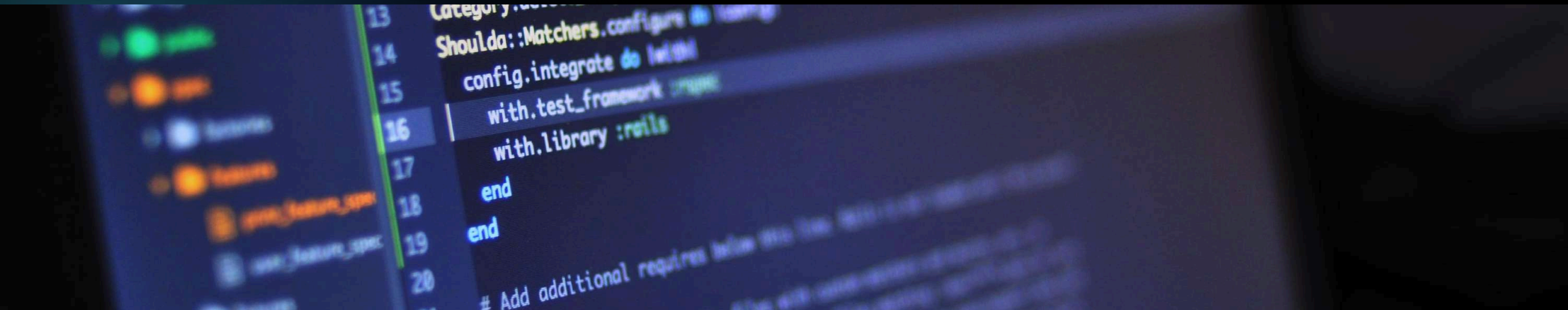
```
<div class="container">
  <div class="row">
    <div class="col-md-6 col-lg-8"> <!-- _____
      <nav id="nav" role="navigation">
        <ul>
          <li><a href="index.html">Home</a></li>
          <li><a href="home-events.html">Home
          <li><a href="multi-col-menu.html">M
          <li class="has-children"> <a href="#"
            <ul>
              <li><a href="tall-button-hea
              <li><a href="image-logo.html
              <li class="active"><a href="#"
            </ul>
          </li>
          <li class="has-children"> <a href="#"
            <ul>
              <li><a href="variable-width-
            </ul>
          </li>
        </ul>
      </nav>
    </div>
  </div>
</div>
```

EXISTING SYSTEM

Current Solutions Used:

- Manual testing (Burp Suite, OWASP ZAP)
- Penetration testing teams
- Standalone scanners (Acunetix, Nessus, SQLMap)
- Educational platforms (OWASP WebGoat, DVWA)

```
17 string sInput;  
18 int iLength, iN;  
19 double dblTemp;  
20 bool again = true;  
21  
22 while (again) {  
23     iN = -1;  
24     again = false;  
25     getline(cin, sInput);  
26     system("cls");  
27     stringstream(sInput) >> dblTemp;  
28     iLength = sInput.length();  
29     if (iLength < 4) {  
30         again = true;  
31         continue;  
32     }  
33     if (sInput[iLength - 3] != '.') {
```



DISADVANTAGES OF EXISTING SYSTEM

- Very expensive tools and professional testing
- High complexity and steep learning curve
- Separate tools for each vulnerability
- Time-consuming manual testing
- Not easily accessible or cloud-based
- No real-time feedback or automated logging

PROPOSED SYSTEM

A unified, web-based platform offering:

- Integrated testing for 5 vulnerabilities
- Real-time detection and results
- Flask backend + responsive frontend
- Supabase cloud database
- JWT authentication with secure user accounts
- Dashboard with analytics, history, and OWASP coverage
- Automated result logging

```
out << "Enter rows and columns for second matrix: " << endl;
cin >> r2 >> c2;

// Taking elements of first matrix.
out << endl << "Enter elements of matrix 1:" << endl;
for(i = 0; i < r1; ++i)
    for(j = 0; j < c1; ++j)
    {
        out << "Enter element a" << i + 1 << j + 1 << " : ";
        cin >> a[i][j];
    }

// Taking elements of second matrix.
out << endl << "Enter elements of matrix 2:" << endl;
for(i = 0; i < r2; ++i)
    for(j = 0; j < c2; ++j)
    {
        out << "Enter element b" << i + 1 << j + 1 << " : ";
        cin >> b[i][j];
    }
```

PROGRAMMING FOR INSIGHTS

- Cost-effective and open-source
- Simple, user-friendly interface
- All tests in one dashboard
- Instant, real-time results
- Strong educational value
- Scalable cloud architecture
- Accessible from any device
- CI/CD integration ready
- Secure design following OWASP standards

```
resp_iter = self.stub.GetDatacenterStatuses()
statuses = {}
for data in resp_iter:
    status = Status(
        status_id=data.id, name=data.name
    )
    statuses[status.name] = status
return statuses
```

SOFTWARE REQUIREMENTS



Backend:

- Python 3.9+
- Flask, Flask-CORS, Flask-Bcrypt, Flask-JWT-Extended
- Supabase Python SDK

ntend:

HTML5, CSS3, JavaScript (ES6)

abase:

Supabase PostgreSQL (cloud)

ironment Variables (.env):

PORT

JWT_SECRET

SUPABASE_URL & SUPABASE_KEY

CORS settings

DEVELOPMENT & DEPLOYMENT

Tools:

- VS Code / PyCharm
- Git
- Postman
- Browser dev tools

Hosting Options:

- Backend: Heroku / AWS / GCP / Local
- Frontend: Served via Flask or Netlify
- Database: Supabase Cloud



The Multi-Layer Security Testing Dashboard provides a modern, unified, cloud-based solution for testing major web vulnerabilities. It eliminates the cost, complexity, and fragmentation seen in existing tools. With real-time detection, automated logging, and a secure architecture, it acts as both a learning platform and a practical security testing tool, helping developers strengthen their application security effectively.

If you want, I can also:

CONCLUSION



THANK YOU
THANK YOU

VINOTH KARTHIK (PRK25DS1012)

GOKULRAJ (PRK25DS1018)

PRASAD (PRK25DS1015)